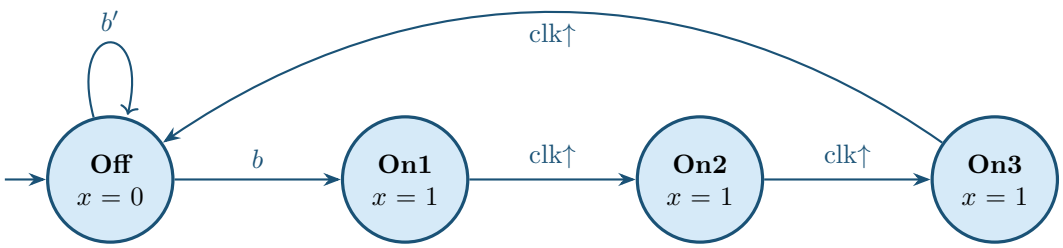
Finite-State Machine (FSM) Design Guide

A Complete Learning Reference



Based on *Digital Design* by Frank Vahid — Chapter 3, Part 2

Sequential Logic Design: Controllers

March 5, 2026

0 Contents

1 Why Do We Need FSMs? 2
1.1 The Problem with Trial-and-Error . . . 2

2 What is a Finite-State Machine (FSM)? 2
2.1 The 5 Components of Every FSM . . . 3

3 Example 1 — Simple Toggle 3
3.1 Problem . . . 3
3.2 FSM State Diagram . . . 3
3.3 Transition Table . . . 4

4 Example 2 — Three-Cycles-High Laser Timer (Full Walkthrough) 4
4.1 Problem Statement . . . 4
4.2 Step 1: Identify the States . . . 4
4.3 Step 2: Draw the FSM State Diagram . . . 5
4.4 Cycle-by-Cycle Operation . . . 5
4.5 Timing Diagram . . . 6

5 The 5-Step Controller Design Process 6
5.1 Overview of Steps . . . 6
5.2 The Standard Architecture . . . 6

6 Step-by-Step: Converting Laser Timer FSM to Circuit 7
6.1 Step 2B: Encode the States . . . 7
6.2 Step 2C: Fill the Truth Table . . . 7
6.3 Step 2D: Derive Boolean Equations . . . 8
6.4 K-Map Derivation (Karnaugh Maps) . . . 9
6.5 Summary of All D Flip-Flop Input Equations . . . 11
6.6 Complete Hardware Circuit with D Flip-Flops . . . 11
6.7 Verification: Trace Through the Circuit . . . 12
6.8 Hardware Connection Summary . . . 13

7 Example 3 — Call/Cancel Light Controller 14
7.1 Problem . . . 14
7.2 FSM State Diagram . . . 14
7.3 State Encoding & Truth Table . . . 14
7.4 D Flip-Flop Equation & Hardware Circuit . . . 15

8 Output Glitching & Solutions 16
8.1 What is Glitching? . . . 16
8.2 Why It Happens . . . 16
8.3 Solution 1: Registered Outputs . . . 16
8.4 Solution 2: State Variables as Outputs . . . 16
8.5 Comparison of Solutions . . . 17

9 Example 4 — Pacemaker (Real-World FSM) 17
9.1 What Is a Pacemaker? 17
9.2 Signals 17
9.3 FSM State Diagram 18
9.4 How It Works 18

10 Chapter Summary & Key Takeaways 18

11 Quick Reference Cheat Sheet 19
11.1 FSM Design Checklist 19
11.2 Common FSM Patterns 19
11.3 Key Formulas 19

12 Practice Problems 20
12.1 Problem 1: 2-Cycle Delay 20
12.2 Problem 2: Sequence Detector 20
12.3 Problem 3: Vending Machine 20

1 Why Do We Need FSMs?

1.1 The Problem with Trial-and-Error

When designing **combinational circuits**, engineers have:

1. A **formal way** to describe behavior — Boolean equations or truth tables
2. A **well-defined process** to convert that behavior into a circuit

But for **sequential circuits** (circuits with memory/state), no such formal tool existed in our discussion — until now.

Why Trial-and-Error Fails

Example: You need a laser timer with output $x = 1$ for exactly 3 clock cycles after a button press.

- Can you "guess" a circuit? Maybe with 3 flip-flops and an OR gate...
- But what if someone presses the button **again** while $x = 1$?
- What about circuits with 9 states? 100 states?
- A guessed circuit may have **undesired behavior** that's hard to detect.

Solution: We need **FSMs** — a formal mathematical model for sequential behavior.

2 What is a Finite-State Machine (FSM)?

Definition

An FSM is a mathematical model that describes sequential circuit behavior by listing all possible **states** and the **transitions** between them, triggered by inputs and clock edges.

2.1 The 5 Components of Every FSM

Component	Symbol	Description
States	◯	All possible conditions the machine can be in
Inputs	$b, x, \ldots$	External signals that affect transitions
Outputs	$x, L, \ldots$	Signals produced in each state
Transitions	$\rightarrow$	Rules for moving from one state to another
Initial State	$\rightarrow$◯	The state on power-up (arrow from nowhere)

Table 1: Five components of a Finite-State Machine

State Count & Flip-Flop Count

n flip-flops $\Rightarrow 2^n$ possible states.

States	Flip-Flops	States	Flip-Flops
2	1	5–8	3
3–4	2	9–16	4

Formula: Flip-flops needed $= \lceil \log_2(\#\text{ states}) \rceil$

3 Example 1 — Simple Toggle

3.1 Problem

Design a circuit whose output x **toggles** (switches between 0 and 1) on every rising clock edge.

3.2 FSM State Diagram

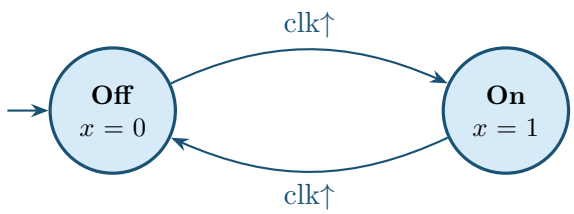


Figure 1: FSM for a simple toggle circuit

3.3 Transition Table

Current State	Clock Edge	Next State	Output x
Off	Rising ↑	On	$0 \to 1$
On	Rising ↑	Off	$1 \to 0$

- Only **2 states** ⇒ need **1 flip-flop** ($2^1 = 2$)
- Output x simply equals the state bit s_0
- Boolean equation: $x = s_0, \;\; n_0 = s_0'$ (complement — toggle)

4 Example 2 — Three-Cycles-High Laser Timer (Full Walkthrough)

4.1 Problem Statement

Laser Timer Problem

A patient lies under a laser and presses button b. The laser (x) must turn **ON** for **exactly 3 clock cycles**, then turn OFF. It stays off until b is pressed again.
Inputs: b (button) **Outputs:** x (laser)

4.2 Step 1: Identify the States

State	Meaning	Output x
Off	Laser is off, waiting for button	0
On1	Laser on — 1st cycle	1
On2	Laser on — 2nd cycle	1
On3	Laser on — 3rd cycle	1

Table 2: States for the three-cycles-high laser timer

4.3 Step 2: Draw the FSM State Diagram

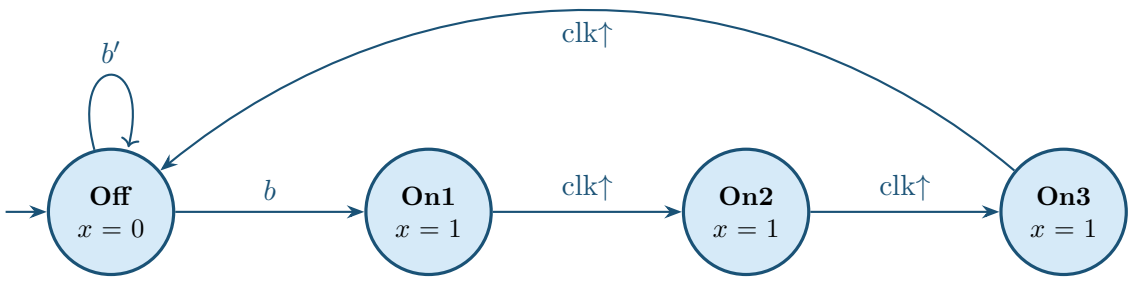


Figure 2: FSM state diagram for the three-cycles-high laser timer

4.4 Cycle-by-Cycle Operation

Clock Cycle	1	2	3	4	5	6	7	8
Button b	0	red!15**1**	0	0	0	0	red!15**1**	0
State	Off	Off	green!15On1	green!15On2	green!15On3	Off	Off	green!15On1
Output x	0	0	green!15**1**	green!15**1**	green!15**1**	0	0	green!15**1**

Table 3: Cycle-by-cycle trace of the laser timer

Key Observation

The button press at cycle 2 is **captured on the rising clock edge**. The state transition to On1 happens at the **start of cycle 3**, not cycle 2!

4.5 Timing Diagram

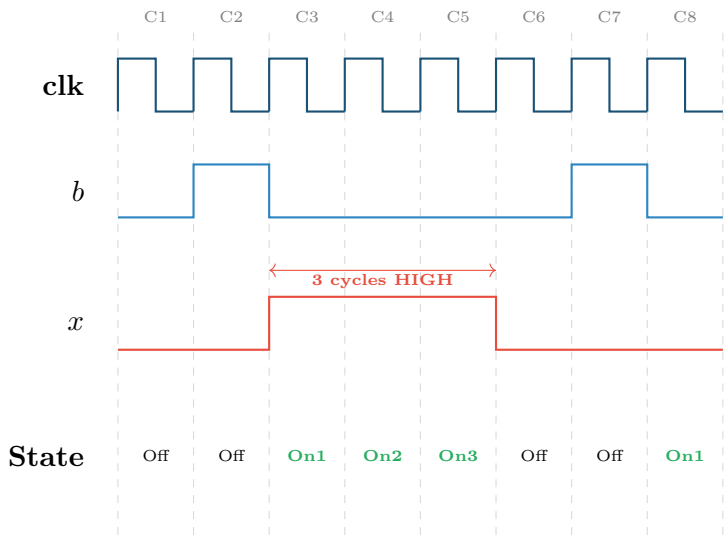


Figure 3: Timing diagram for the laser timer

5 The 5-Step Controller Design Process

This is the **core methodology** for converting any FSM into a working digital circuit.

5.1 Overview of Steps

Step	Name	What You Do
1	**Capture the FSM**	Draw the FSM state diagram describing desired behavior
2A	**Set Up Architecture**	Use state register (D flip-flops) + combinational logic block
2B	**Encode the States**	Assign a unique binary number to each state
2C	**Fill the Truth Table**	Translate all FSM transitions into truth table rows
2D	**Implement Comb. Logic**	Derive Boolean equations from truth table, build gates

Table 4: The 5-step controller design process (Table 3.2 from textbook)

5.2 The Standard Architecture

Every FSM controller uses the **same** architecture:

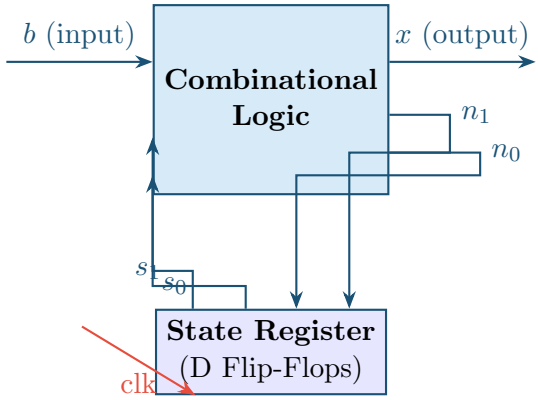


Figure 4: Standard FSM controller architecture

How It Works — Every Clock Cycle

1. On each **rising clock edge**, the state register captures n_1, n_0 and stores them as s_1, s_0
2. The combinational logic reads **current state** (s_1, s_0) and **input** (b)
3. It computes the **output** (x) and the **next state** (n_1, n_0)
4. Repeat!

6 Step-by-Step: Converting Laser Timer FSM to Circuit

6.1 Step 2B: Encode the States

State	s_1	s_0	Binary Encoding
Off	0	0	00
On1	0	1	01
On2	1	0	10
On3	1	1	11

Table 5: State encoding for laser timer

6.2 Step 2C: Fill the Truth Table

For each combination of current state (s_1, s_0) and input (b), determine output (x) and next state (n_1, n_0):

State	Inputs			Outputs			Explanation
	s_1	s_0	b	x	n_1	n_0	
Off	0	0	0	0	0	0	b=0: stay in Off
	0	0	1	0	0	1	b=1: go to On1
On1	0	1	0	1	1	0	always → On2
	0	1	1	1	1	0	always → On2
On2	1	0	0	1	1	1	always → On3
	1	0	1	1	1	1	always → On3
On3	1	1	0	1	0	0	always → Off
	1	1	1	1	0	0	always → Off

Table 6: Complete truth table for the laser timer controller

How to Read Each Row

Row 2: Current state = Off (s_1=0, s_0=0), button pressed (b=1).
From FSM: Off → On1 when b=1. On1's encoding is `01`, so n_1=0, n_0=1.
Off's output: x=0.

Row 3: Current state = On1 (s_1=0, s_0=1), b=0.
From FSM: On1 → On2 (unconditional). On2's encoding is `10`, so n_1=1, n_0=0.
On1's output: x=1.

6.3 Step 2D: Derive Boolean Equations

Output x:

From the truth table, $x = 1$ in rows 3–8 (all states except Off):

$$x = \overline{s_1} \cdot s_0 + s_1 \cdot \overline{s_0} + s_1 \cdot s_0$$
$$x = s_0 + s_1$$

$$\boxed{x = s_1 + s_0} \tag{1}$$

Output is 1 whenever we're NOT in the Off state (00).

Next State n_1:

$n_1 = 1$ when state is On1 (s_1=0, s_0=1) or On2 (s_1=1, s_0=0), regardless of b:

$$n_1 = \overline{s_1} \cdot s_0 + s_1 \cdot \overline{s_0}$$

$$\boxed{n_1 = s_1 \oplus s_0} \tag{2}$$

This is the XOR of the two state bits.

Next State n_0:

$n_0 = 1$ when: Off with b=1, or On2 (regardless of b):

$$\begin{aligned}
n_0 &= \overline{s_1} \cdot \overline{s_0} \cdot b + s_1 \cdot \overline{s_0} \cdot \overline{b} + s_1 \cdot \overline{s_0} \cdot b \\
n_0 &= \overline{s_1} \cdot \overline{s_0} \cdot b + s_1 \cdot \overline{s_0} \\
n_0 &= \overline{s_0} \cdot (\overline{s_1} \cdot b + s_1) \\
n_0 &= \overline{s_0} \cdot (s_1 + b) \quad (\text{using } A + \overline{A} \cdot B = A + B)
\end{aligned}$$

$$\boxed{n_0 = \overline{s_0} \cdot (s_1 + b)} \tag{3}$$

6.4 K-Map Derivation (Karnaugh Maps)

Now we use **Karnaugh Maps** to systematically derive minimized Boolean equations from the truth table. For a D flip-flop, the key rule is:

D Flip-Flop Rule

For D flip-flops, the **D input** equals the **next state**:

$D_1 = n_1$ (what gets loaded into flip-flop 1 at next clock edge)
$D_0 = n_0$ (what gets loaded into flip-flop 0 at next clock edge)

So our job is to find the equations for D_1 and D_0 from the truth table.

K-Map for Output x

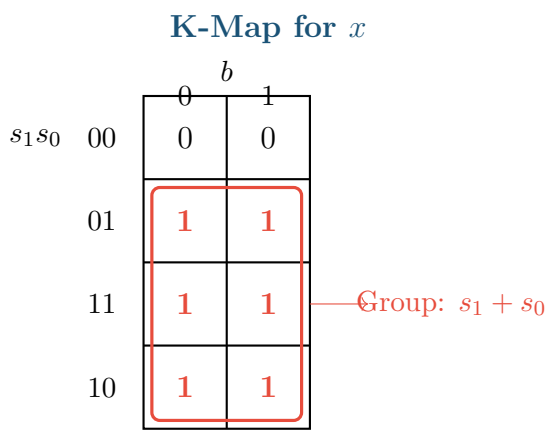


Figure 5: K-Map for output x — all non-Off states give $x = 1$

$$\boxed{x = s_1 + s_0} \tag{4}$$

Meaning: Output $x = 1$ whenever the FSM is NOT in state Off ($s_1 s_0 = 00$).

K-Map for D_1 (= n_1, D input of flip-flop 1)

K-Map for $D_1 = n_1$

$s_1 s_0$ \ b	0	1
00	0	0
01	1	1
11	0	0
10	1	1

Group 1: $\overline{s_1} \cdot s_0$

Group 2: $s_1 \cdot \overline{s_0}$

Figure 6: K-Map for D_1 — two separate groups that cannot be combined

The two groups are **not adjacent** in the K-Map, so they cannot be combined further:

$$D_1 = \overline{s_1} \cdot s_0 + s_1 \cdot \overline{s_0} \tag{5}$$

$$\boxed{D_1 = s_1 \oplus s_0 \qquad \text{(XOR gate)}} \tag{6}$$

Why XOR?

$\overline{A} \cdot B + A \cdot \overline{B}$ is the **definition** of XOR ($A \oplus B$).
So D_1 needs just **one XOR gate**!

K-Map for D_0 (= n_0, D input of flip-flop 0)

K-Map for $D_0 = n_0$

$s_1 s_0$ \ b	0	1
00	0	1
01	0	0
11	0	0
10	1	1

Group 1: $\overline{s_1} \cdot \overline{s_0} \cdot b$

Group 2: $s_1 \cdot \overline{s_0}$

Figure 7: K-Map for D_0 — one pair and one single cell

From the K-Map:

$$D_0 = \overline{s_1} \cdot \overline{s_0} \cdot b \ + \ s_1 \cdot \overline{s_0} \tag{7}$$

$$D_0 = \overline{s_0} \cdot (\overline{s_1} \cdot b + s_1) \tag{8}$$

$$D_0 = \overline{s_0} \cdot (s_1 + b) \quad \text{(using Boolean identity: } A + \overline{A}B = A + B) \tag{9}$$

$$\boxed{D_0 = \overline{s_0} \cdot (s_1 + b)} \tag{10}$$

Simplification Step-by-Step

1. Start: $\overline{s_1} \cdot \overline{s_0} \cdot b + s_1 \cdot \overline{s_0}$
2. Factor out $\overline{s_0}$: $\overline{s_0} \cdot (\overline{s_1} \cdot b + s_1)$
3. Apply $A + \overline{A}B = A + B$ where $A = s_1$, $B = b$: $\overline{s_0} \cdot (s_1 + b)$

This requires: **1 NOT gate + 1 OR gate + 1 AND gate**

6.5 Summary of All D Flip-Flop Input Equations

Complete Design Equations

These are the **final equations** needed to build the hardware:

Signal	Equation	Hardware Needed
Output:	$x = s_1 + s_0$	1 OR gate
FF1 input:	$D_1 = s_1 \oplus s_0$	1 XOR gate
FF0 input:	$D_0 = \overline{s_0} \cdot (s_1 + b)$	1 NOT + 1 OR + 1 AND gate

6.6 Complete Hardware Circuit with D Flip-Flops

Below is the **complete circuit** showing exactly how to wire two D flip-flops with the combinational logic gates:

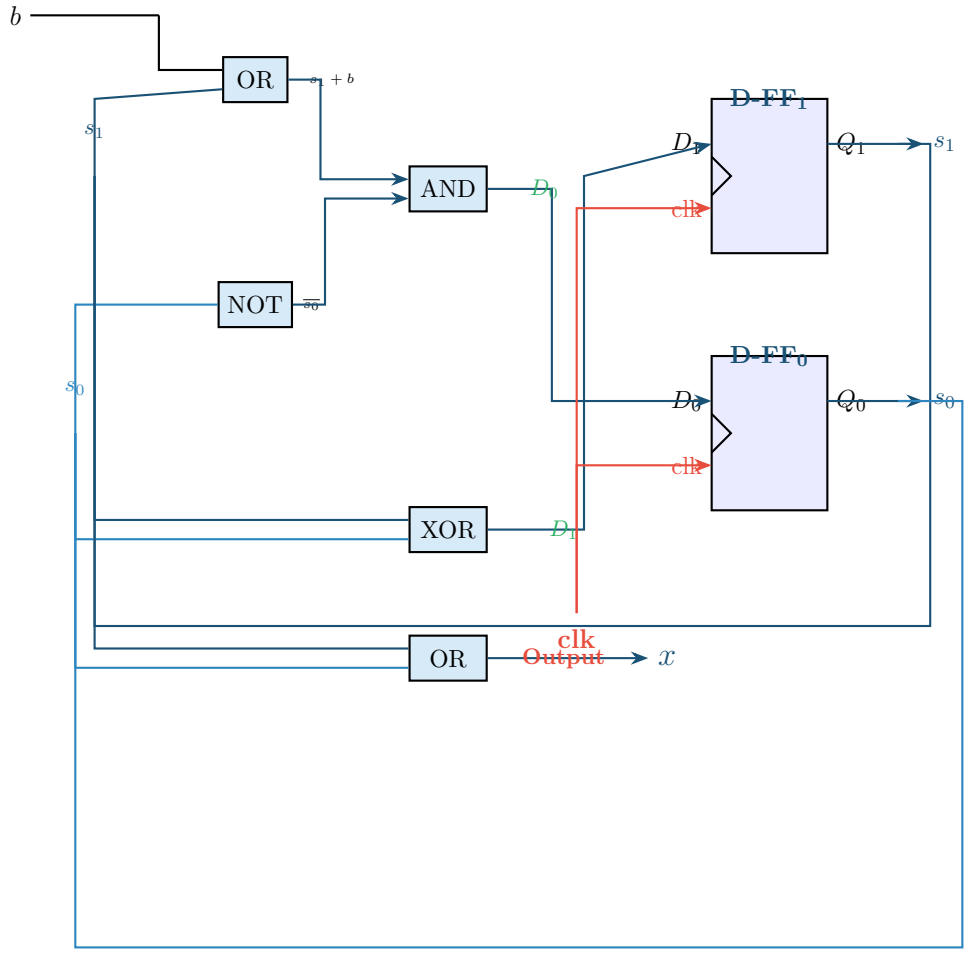


Figure 8: Complete hardware circuit for the three-cycles-high laser timer using D flip-flops

Circuit Component Count

Component	Qty	Purpose
D Flip-Flop	2	Store s_1 (FF$_1$) and s_0 (FF$_0$)
OR gate	2	One for $x = s_1 + s_0$, one for $s_1 + b$ in D_0
XOR gate	1	For $D_1 = s_1 \oplus s_0$
AND gate	1	For $D_0 = \overline{s_0} \cdot (s_1 + b)$
NOT gate	1	For $\overline{s_0}$ in D_0
Total	**7**	**2 FFs + 5 gates**

6.7 Verification: Trace Through the Circuit

Let's verify the circuit by tracing it through each state:

State	Current		b	Gate Outputs			Next Clk Edge		Correct?
	s_1	s_0		x	D_1	D_0	s_1'	s_0'	
Off	0	0	0	0	0	0	0	0	✓ Off
Off	0	0	1	0	0	1	0	1	✓ On1
On1	0	1	–	1	1	0	1	0	✓ On2
On2	1	0	–	1	1	1	1	1	✓ On3
On3	1	1	–	1	0	0	0	0	✓ Off

Table 7: Circuit verification — traces match the FSM exactly

Let's verify one row **manually**:

Manual Trace: State

1. $x = s_1 + s_0 = 1 + 0 = 1$ ✓ (laser ON)
2. $D_1 = s_1 \oplus s_0 = 1 \oplus 0 = 1$
3. $D_0 = \overline{s_0} \cdot (s_1 + b) = \overline{0} \cdot (1 + b) = 1 \cdot 1 = 1$
4. At next clock edge: $s_1 \leftarrow D_1 = 1, \;\; s_0 \leftarrow D_0 = 1$
5. New state: $s_1 s_0 = 11 = \text{On3}$ ✓

6.8 Hardware Connection Summary

For breadboard or schematic implementation, here is the complete wiring list:

#	From	To	Signal
1	Input b	OR_1 pin A	External button
2	FF_1 Q output	OR_1 pin B	Feedback s_1
3	OR_1 output	AND pin A	$s_1 + b$
4	FF_0 Q output	NOT input	Feedback s_0
5	NOT output	AND pin B	$\overline{s_0}$
6	AND output	FF_0 D input	$D_0 = \overline{s_0} \cdot (s_1 + b)$
7	FF_1 Q output	XOR pin A	Feedback s_1
8	FF_0 Q output	XOR pin B	Feedback s_0
9	XOR output	FF_1 D input	$D_1 = s_1 \oplus s_0$
10	FF_1 Q output	OR_2 pin A	s_1
11	FF_0 Q output	OR_2 pin B	s_0
12	OR_2 output	Output x	$x = s_1 + s_0$ (laser)
13	Clock oscillator	FF_1 clk & FF_0 clk	System clock

Table 8: Complete wiring list for hardware implementation

7 Example 3 — Call/Cancel Light Controller

7.1 Problem

A hospital call light system: pressing **Call** turns the light ON; pressing **Cncl** turns it OFF. If both pressed simultaneously, the light **stays in its current state**.

Inputs: Call, Cncl **Output:** L (1 = light ON)

7.2 FSM State Diagram

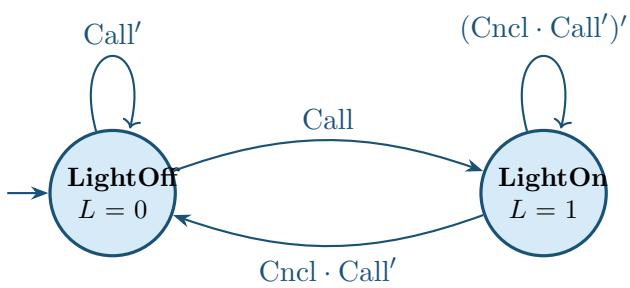


Figure 9: FSM for hospital call/cancel light controller

7.3 State Encoding & Truth Table

2 states ⇒ 1 flip-flop: LightOff = 0, LightOn = 1

s_0	Call	Cncl	L	n_0	Explanation
0	0	0	0	0	No call → stay off
0	0	1	0	0	Cancel while off → stay off
0	1	0	0	1	Call pressed → turn on
0	1	1	0	1	Call overrides → turn on
1	0	0	1	1	No input → stay on
1	0	1	1	0	Cancel → turn off
1	1	0	1	1	Call while on → stay on
1	1	1	1	1	Both pressed → stay on

Table 9: Truth table for the call/cancel light controller

7.4 D Flip-Flop Equation & Hardware Circuit

Since there are only **2 states**, we need **1 D flip-flop**:

$$L = s_0 \quad \text{(output directly from FF)} \tag{11}$$

$$n_0 = \overline{s_0} \cdot \text{Call} + s_0 \cdot \overline{\text{Cncl}} + s_0 \cdot \text{Call} \tag{12}$$

Simplifying using $s_0 \cdot \overline{\text{Cncl}} + s_0 \cdot \text{Call} = s_0 \cdot (\overline{\text{Cncl}} + \text{Call})$, and then $\overline{s_0} \cdot \text{Call} + s_0 \cdot \text{Call} = \text{Call}$:

$$\boxed{D_0 = n_0 = \text{Call} + s_0 \cdot \overline{\text{Cncl}}} \tag{13}$$

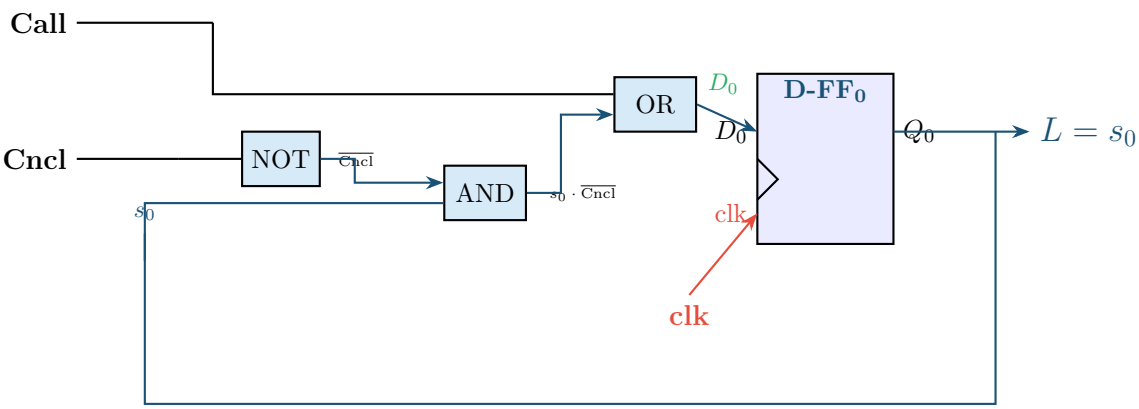


Figure 10: Complete hardware circuit for call/cancel light using 1 D flip-flop

Component Count

1 D-FF + 1 NOT gate + 1 AND gate + 1 OR gate = 4 components total

8 Output Glitching & Solutions

8.1 What is Glitching?

Glitching Definition

Glitching = temporary incorrect output values that appear *briefly* when the combinational logic computes new outputs after a state change, before stable values are reached.

8.2 Why It Happens

When the state register changes (e.g., from `01` to `10`), the bits don't switch at **exactly the same instant**:

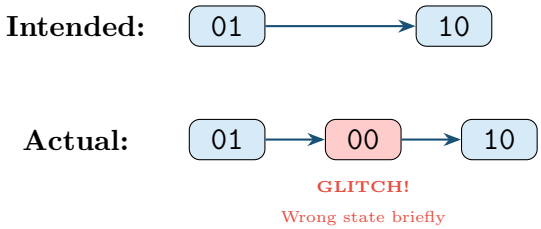


Figure 11: How glitching occurs during state transitions

8.3 Solution 1: Registered Outputs

Add a **D flip-flop** on each output to capture it only at the clock edge (when stable):

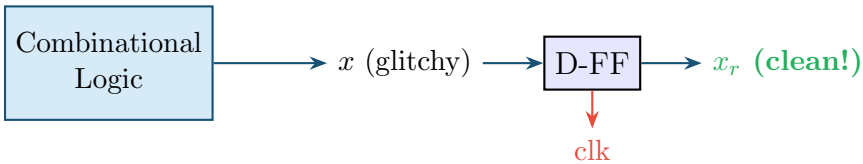


Figure 12: Registered output solution — adds 1 clock cycle delay but removes glitches

Trade-off: Output is delayed by **one clock cycle**.

8.4 Solution 2: State Variables as Outputs

Choose state encoding so the **state bits themselves** serve as outputs:

State	s_2	s_1	s_0	Output x
Off	0	0	0	$0 = s_0$
On1	0	1	1	$1 = s_0$
On2	1	0	1	$1 = s_0$
On3	1	1	1	$1 = s_0$

Table 10: State-as-output encoding: $x = s_0$ directly from flip-flop

Since $x = s_0$ comes **directly from a flip-flop**, it is **glitch-free** without any delay!

8.5 Comparison of Solutions

Method	Glitch-Free?	Extra Hardware	Output Delay
No fix	×	None	None
Registered outputs	✓	1 D-FF per output	+1 clock cycle
State-as-output	✓	May need more FFs	None

Table 11: Comparison of glitching solutions

9 Example 4 — Pacemaker (Real-World FSM)

9.1 What Is a Pacemaker?

A small device implanted in the chest that sends electrical pulses to keep the heart beating regularly.

9.2 Signals

Signal	Type	Meaning
s	Input	Heart sensor (1 = heartbeat detected)
z	Input	Timer expired (1 = time's up, no beat detected)
t	Output	Timer control (1 = reset/start timer)
p	Output	Pace pulse (1 = send electrical pulse to heart)

9.3 FSM State Diagram

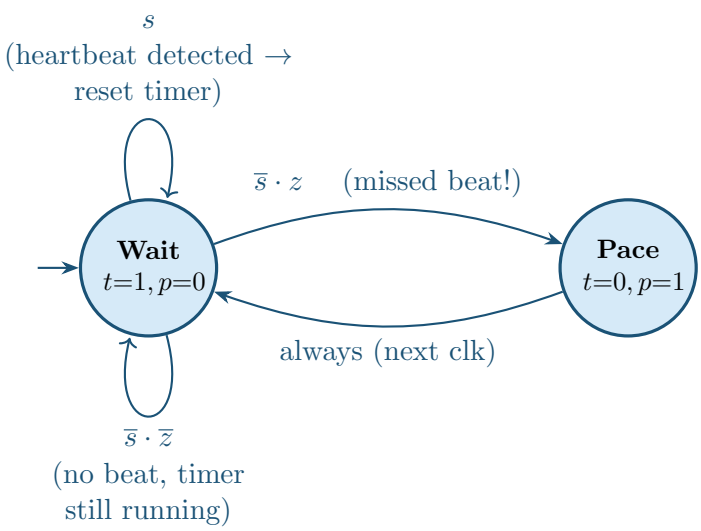


Figure 13: Pacemaker controller FSM

9.4 How It Works

1. **Wait state** (t=1, p=0): Timer is running, no pacing.
   - If heartbeat detected (s=1): Stay in Wait, reset timer — heart is fine!
   - If timer expires (z=1) without heartbeat (s=0): Heart missed a beat! → Go to Pace
2. **Pace state** (p=1, t=0): Send one electrical pulse.
   - Immediately return to Wait on next clock edge.

Safety-Critical Design

This is a **life-critical** FSM! A bug could harm a patient. This demonstrates why **formal FSM design** (not trial-and-error) is absolutely essential for safety-critical systems.

10 Chapter Summary & Key Takeaways

What We Learned

1. **Sequential circuits have state** — they remember past inputs.
2. **D flip-flop** is the robust bit-storage device; several together form a **register**.
3. **FSM** is the formal model for sequential behavior (like Boolean equations are for combinational).
4. The **5-step design process** systematically converts any FSM to a working

circuit:

(a) Capture FSM → 2A. Architecture → 2B. Encode → 2C. Truth Table → 2D. Logic

5. **Glitching** can be solved via registered outputs or state-as-output encoding.
6. We can now build the class of sequential circuits known as **controllers**.

11 Quick Reference Cheat Sheet

11.1 FSM Design Checklist

Step 1: Draw FSM state diagram

- Identify all states, inputs, outputs
- Draw transitions with conditions, mark initial state

Step 2A: Set up architecture: `inputs` → `[Comb Logic]` → `outputs`, with `[State Register]` below

Step 2B: Encode states with unique binary codes

Step 2C: Create truth table (inputs: state bits + FSM inputs; outputs: FSM outputs + next state bits)

Step 2D: Derive Boolean equations, simplify, draw gates

11.2 Common FSM Patterns

Pattern	Use Case	Example
Counter	Do something for N cycles	Laser timer
Toggle	Alternate between two states	Clock divider
Wait-Act	Wait for trigger, act, return	Pacemaker
Sequence Detector	Detect input pattern	Security code

11.3 Key Formulas

$$\text{Number of flip-flops needed} = \lceil \log_2(\text{number of states}) \rceil$$

2 states → 1 FF	3–4 states → 2 FFs
5–8 states → 3 FFs	9–16 states → 4 FFs

12 Practice Problems

12.1 Problem 1: 2-Cycle Delay

Design an FSM that outputs $y = 1$ exactly **2 clock cycles** after input a becomes 1.

Hint

3 states: Wait, Delay1, Delay2. When $a = 1$: Wait $\rightarrow$ Delay1 $\rightarrow$ Delay2 (y=1) $\rightarrow$ Wait.

12.2 Problem 2: Sequence Detector

Design an FSM that outputs $z = 1$ when it detects the input sequence **1, 0, 1** on input w.

Hint

4 states: S_0 (initial), Got1 (seen "1"), Got10 (seen "1,0"), Got101 (seen "1,0,1" $\rightarrow$ z=1). Consider overlapping sequences!

12.3 Problem 3: Vending Machine

A vending machine accepts nickels (N) and dimes (D). It dispenses a drink (output dispense = 1) when 15 cents or more is inserted. Design the FSM.

Hint

States represent accumulated cents: S_0 (0¢), S_5 (5¢), S_{10} (10¢), S_{15} (15+¢, dispense).

Source: Frank Vahid, Digital Design, Chapter 3 — Sequential Logic Design: Controllers

Key Takeaway: FSMs provide a formal, systematic way to design sequential circuits. Always use the 5-step process — never guess!